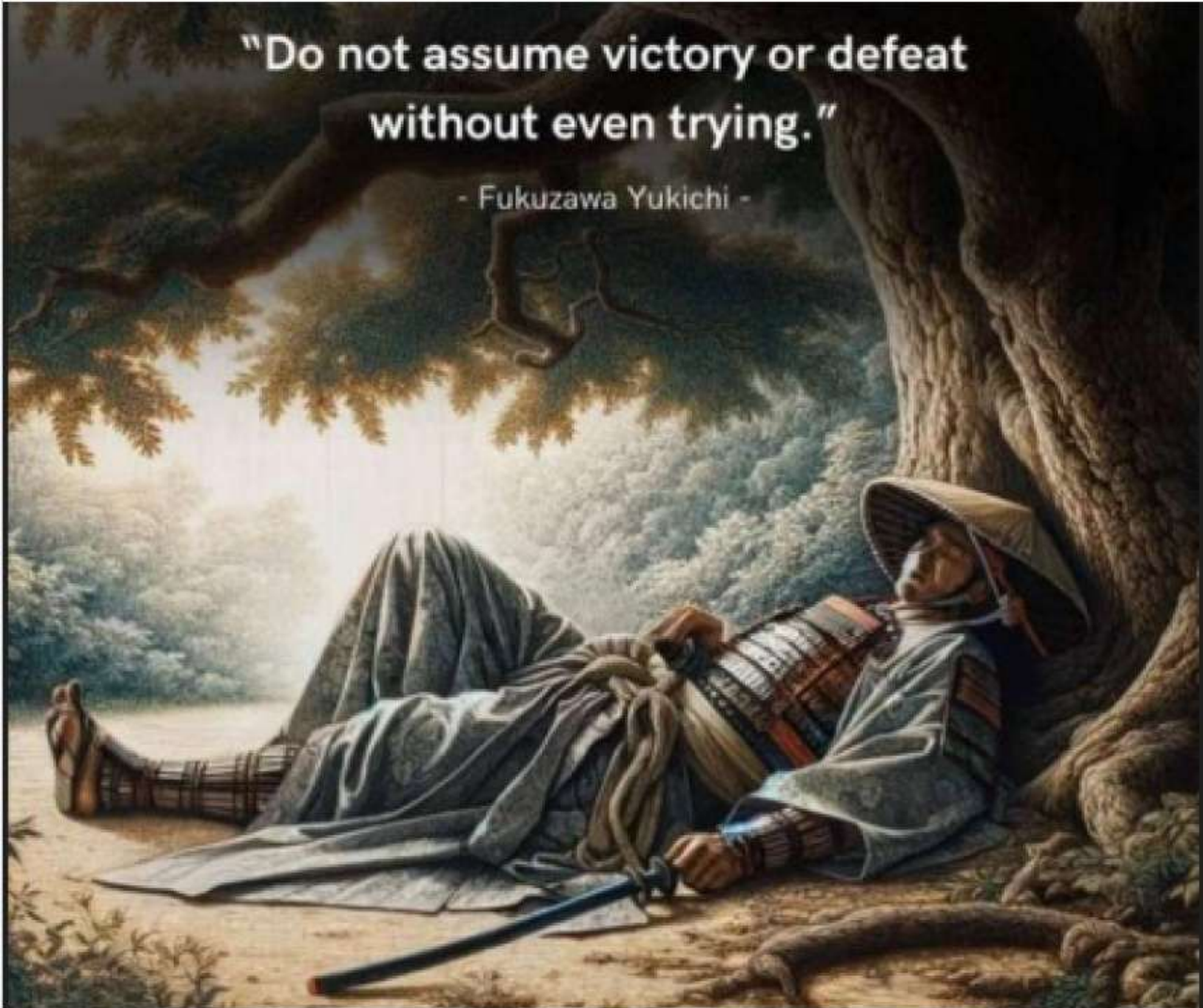


**"Do not assume victory or defeat
without even trying."**

- Fukuzawa Yukichi -





Microcontrollers

Rex Joseph

Department of Electrical and
Electronics Engineering

Hardware, applications, programming

1. **Stand up straight:** Keep your body aligned. Your head should be up, and your back should be straight.
2. **Shift your weight:** Move your weight to one leg to prepare for the first step.
3. **Lift your foot:** Raise one foot, lifting it from the heel. As you do, make sure your knee is slightly bent, and the leg is in line with your body.
4. **Move the foot forward:** As you lift your foot, move it forward in a smooth, controlled motion.
5. **Place the foot on the ground:** Put the heel of the lifted foot down first, then roll the foot to the ball of the foot and finally put your toes on the ground.
6. **Shift your weight again:** Once your foot is planted, shift your body weight onto it.
7. **Repeat:** Now repeat the same process with your other leg.

Hardware, applications, programming

1. **Stand up straight:** Keep your body aligned. Your head should be up, and your back should be straight.
2. **Shift your weight:** Move your weight to one leg to prepare for the first step.
3. **Lift your foot:** Raise one foot, lifting it from the heel. As you do, make sure your knee is slightly bent, and the leg is in line with your body.
4. **Move the foot forward:** As you lift your foot, move it forward in a smooth, controlled motion.
5. **Place the foot on the ground:** Put the heel of the lifted foot down first, then roll the foot to the ball of the foot and finally put your toes on the ground.
6. **Shift your weight again:** Once your foot is planted, shift your body weight onto it.
7. **Repeat:** Now repeat the same process with your other leg.

ChatGPT on “how do I walk?”

Hardware, applications, programming

1. **Stand up straight:** Keep your body aligned. Your head should be up, and your back should be straight.
2. **Shift your weight:** Move your weight to one leg to prepare for the first step.
3. **Lift your foot:** Raise one foot, lifting it from the heel. As you do, make sure your knee is slightly bent, and the leg is in line with your body.
4. **Move the foot forward:** As you lift your foot, move it forward in a smooth, controlled motion.
5. **Place the foot on the ground:** Put the heel of the lifted foot down first, then roll the foot to the ball of the foot and finally put your toes on the ground.
6. **Shift your weight again:** Once your foot is planted, shift your body weight onto it.
7. **Repeat:** Now repeat the same process with your other leg.

ChatGPT on “how do I walk?”



	000	040	080	0C0	100	140	180	1C0	200	240	280	2C0	300	340	380	3C0
0xxx																
4xxx																
8xxx																
Cxxx																
1xxx	RRC	RRC.B	SWPB		RRA	RRA.B	SXT		PUSH	PUSH.B	CALL		RETI			
14xx																
18xx																
1Cxx																
20xx	JNE/JNZ															
24xx	JEQ/JZ															
28xx	JNC															
2Cxx	JC															
30xx	JN															
34xx	JGE															
38xx	JL															
3Cxx	JMP															
4xxx	MOV, MOV.B															
5xxx	ADD, ADD.B															
6xxx	ADDC, ADDC.B															
7xxx	SUBC, SUBC.B															
8xxx	SUB, SUB.B															
9xxx	CMP, CMP.B															
Axxx	DADD, DADD.B															
Bxxx	BIT, BIT.B															
Cxxx	BIC, BIC.B															
Dxxx	BIS, BIS.B															
Exxx	XOR, XOR.B															
Fxxx	AND, AND.B															

Figure 3-12. Core Instruction Map

Addressing Modes

The MSP430 architecture has seven possibilities to address its operands. Four of them are implemented in the CPU, two of them result from the use of the program counter (PC) as a register, and a further one is claimed by indexing a register

The single operand instructions can use all of the seven addressing modes, the double operand instructions can use all of them for the source operand, and four of them for the destination operand.

Double Operand Instructions

Mnemonic	Source, Destination	
		
	Register	Register
	Indexed	Indexed
	Absolute	Absolute
	Symbolic	Symbolic
	Immediate	
	Register indirect	
	Register indirect autoincrement	

Single Operand Instructions

Mnemonic	Destination
	
	Register
	Indexed
	Absolute
	Symbolic
	Immediate
	Register indirect
	Register indirect autoincrement

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Op-code				S-Reg				Ad	B/W	As		D-Reg			

Track the instructions from opcode to the other fields

And also the contents of memory, especially the words following PC

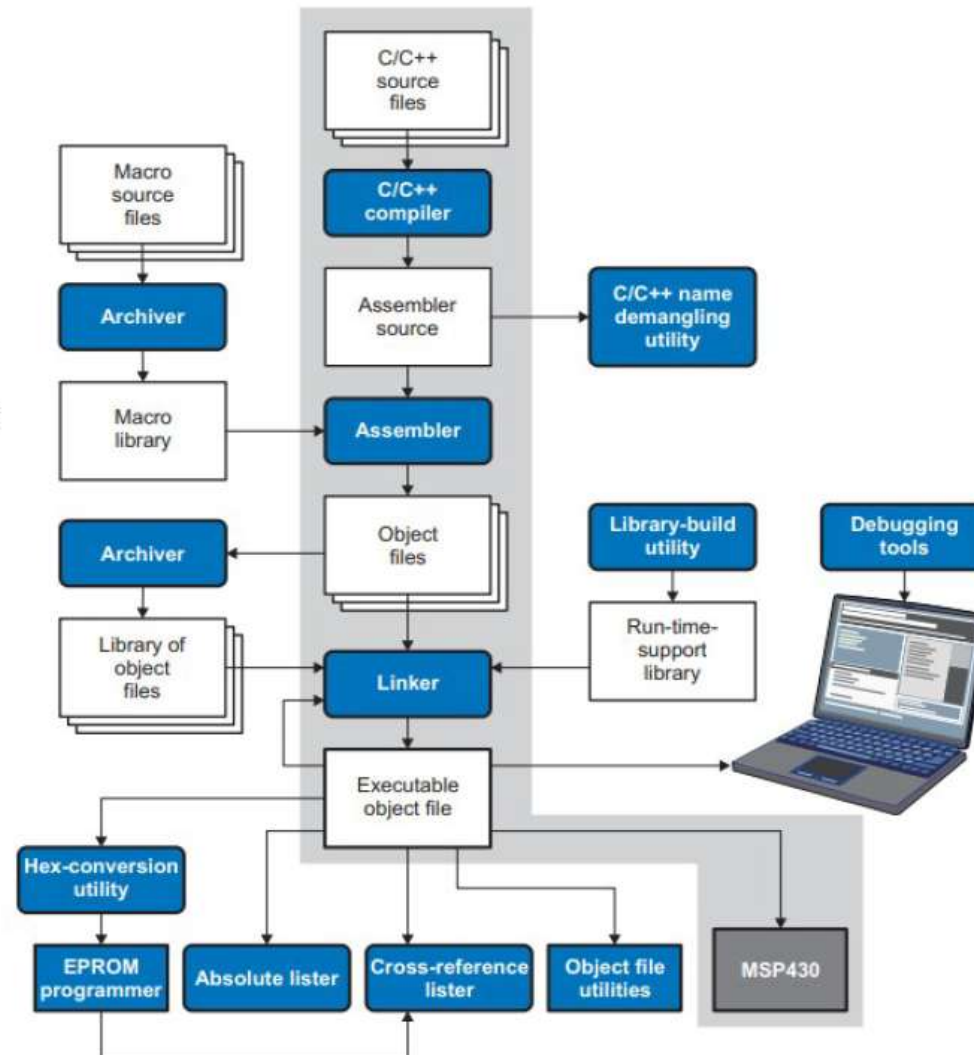
For the different addressing modes (two operand)

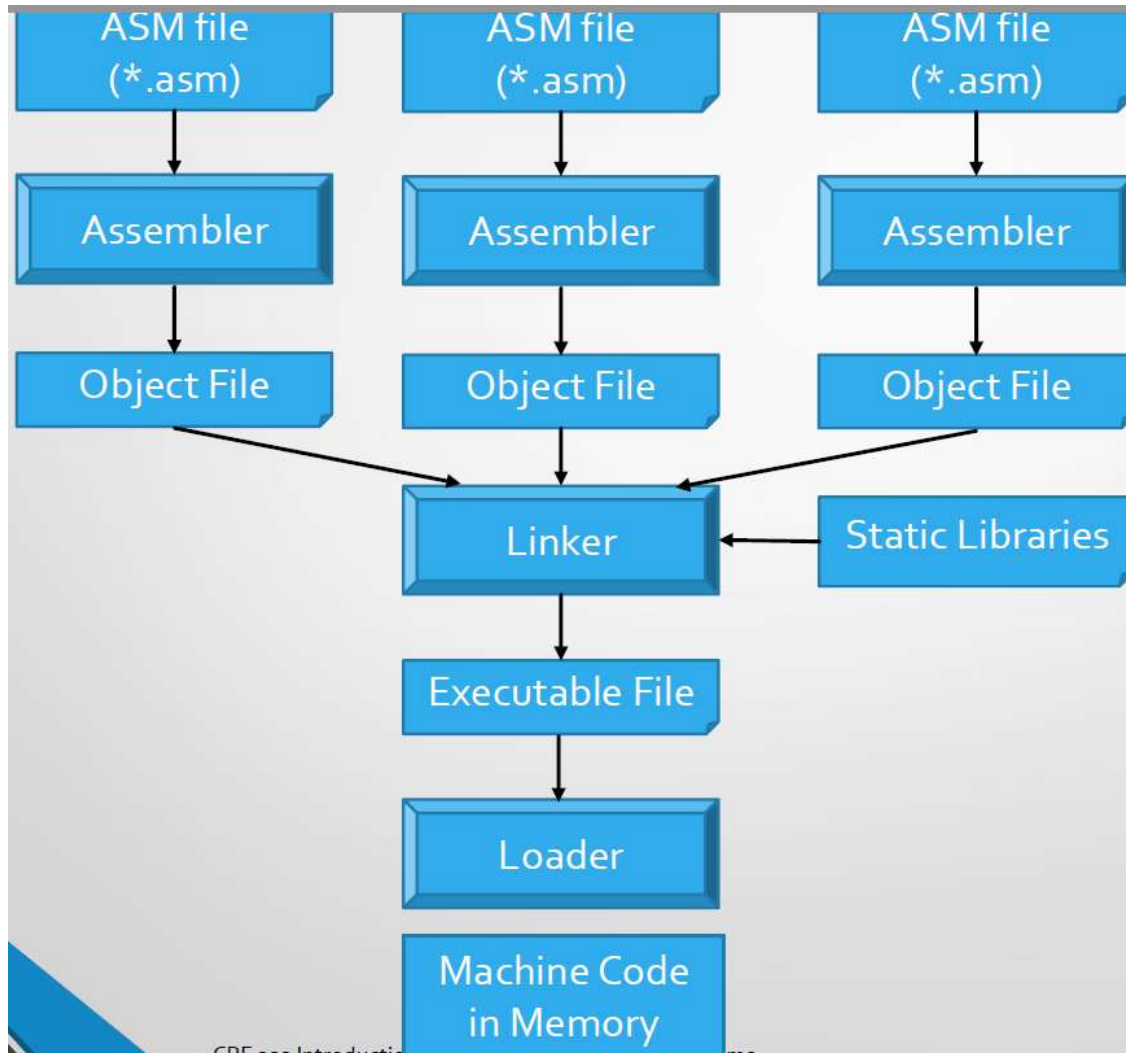
Software Development Tools Overview

MSP430 software development flow.

The shaded portion highlights the most common development path; the other portions are optional.

The other portions are peripheral (auxiliary) functions that enhance the development process.





Object Modules

Object File Format Specifications

The object files created by the assembler and linker conform to the **ELF (Executable and Linking Format)** binary format, which is used by the Embedded Application Binary Interface (EABI).

Executable Object Files

The linker produces executable object modules.

An executable object module has the same format as object files that are used as linker input.

The sections in an executable object module, however, have been combined and placed in target memory, and the relocations are all resolved.

To run a program, the data in the executable object module must be transferred, or **loaded**, into target system memory

Object Modules

The smallest unit of an object file is a section.

A section is a block of code or data that **occupies contiguous space** in the memory map.

Each section of an object file is **separate and distinct**.

ELF format executable object files contain segments. An **ELF segment is a meta-section**.

It represents a contiguous region of target memory and is a collection of sections that have the same property, such as writeable or readable.

An ELF loader needs the segment information, but does not need the section information.

Object Modules

The ELF standard allows the linker to omit ELF section information entirely from the executable object file.

Object files usually contain three default sections:

- .text** section contains executable code
- .data** section usually contains initialized data
- .bss** section usually reserves space for uninitialized variables

Some targets allow content other than text, such as constants, in .text sections.

Object Modules

```
1 *****
2 ; Subroutine      : Increment/decrement Teta_cm
3 ; File           : angle_update.asm
4 ; Date of creation : Feb 12, 2011
5 ; Created by      : Rex Joseph
6 *****
7     .include "reg_2812.h"
8
9 ; Global Variable identification
10
11     .def      _ang_updt      ; subroutine
12
13     .ref      f_rotor, Teta_cm, var_page, cnt_inc
14
15     .text
16 _ang_updt:
17 *****
18 * Update the value of Teta_cm based on the difference between the
19 * reference and the actual electrical equivalent of the rotor
20 * mechanical frequency (both in 8.8 format). The frequency
21 * difference is multiplied by the count increment/Hz (8.8 format)
22 *****
```

Object Modules

```

23      MOVW      DP,#var_page
24      MOV       AL,@f_rotor
25      MOV       AH,#3200h      ; reference freq (50 Hz) in 8.8 format
26      CMP       AH,AL
27      SB        negseq,LT
28      SUB       AH,AL
29      MOV       T,AH
30      MPYU      ACC,T,@cnt_inc ; count increase/Hz for no clk scaling
31      MOV       AL,@Teta_cm
32      ADD       AL,AH
33      MOV       @Teta_cm,AL
34      LRETR
35
36 negseq:
37
38      SUB       AL,AH
39      MOV       T,AL
40      MPYU      ACC,T,@cnt_inc
41      MOV       AL,@Teta_cm
42      SUB       AL,AH
43      MOV       @Teta_cm,AL
44      LRETR

```

Object Modules

```
.bss    var_page,1          ;1.to obtain data page of variables
.bss    tmp,1              ;2.
.bss    tmp1,1            ;3.temporary variable
.bss    mi_max,1          ;4
.bss    Vref1,1           ;4,variable for sector calculation
.bss    Vref2,1           ;5,variable for sector calculation
.bss    Vref3,1           ;6,variable for sector calculation
.bss    sinTeta_cm,1      ;7,sine rotor flux position with current model, 4.12 f
.bss    cosTeta_cm,1      ;8,cosine rotor flux position with current model, 4.12 f
.bss    Teta_cm,1         ;9,real rotor flux position, output
.bss    tetaincr,1        ;10,variable used in current model
.bss    Index,1          ;11,pointer used to access sine look-up table
.bss    da1,1             ;12
.bss    da2,1             ;13
.bss    da3,1             ;14
.bss    da4,1             ;15
.bss    f_rotor,1         ;16
.bss    cnt_inc,1         ;17
.bss    enc_inc,1         ;18
.bss    speedstep,1       ;19
.bss    Va,1
.bss    Vb,1
.bss    Vc,1
```

Loading a Program

A program needs to be placed into the target device's memory before it may be executed.

Loading is the process of preparing a program for execution by initializing device memory with the program's code and data.

A loader might be another program on the device, an external agent (for example, a debugger), or the device might initialize itself after power-on, which is known as **bootstrap loading**, or **bootloading**.

The loader is responsible for constructing the load image in memory before the program starts.

The **load image** is the **program's code and data in memory before execution**.

Loading a Program

What exactly constitutes loading depends on the environment, such as whether an operating system is present.

A program may be loaded in the following ways:

- **A debugger** running on a connected host workstation.

In a typical embedded development setup, the device is subordinate to a host running a debugger such as Code Composer Studio (CCS).

The device is connected with a communication channel such as a **JTAG interface**. CCS reads the program and writes the load image directly to target memory through the communications interface.

- **Another program running on the device.**

The running program can create the load image and transfer control to the loaded program. If an operating system is present, it may have the ability to load and run programs.

Loading a Program

- **"Burning"** the load image onto an EPROM module.

The hex converter (hex430) can assist with this by converting the executable object file into a format suitable for input to an EPROM programmer.

The EPROM is placed onto the device itself and becomes a part of the device's memory.

- Bootstrap loading from a **dedicated peripheral**, such as an I2C peripheral. The device may require a small program called a bootloader to perform the loading from the peripheral.

The hex converter can assist in creating a bootloader

(details of bootloading, primary secondary, boot tabe addresses – XX)

Programming

Machine language

- Description of instructions in terms of 0's and 1's. This is the model for the actual contents in program memory
- Normally presented in Hex Notation.

- Assembly language

- Machine language in human-readable form. Allows better control of program

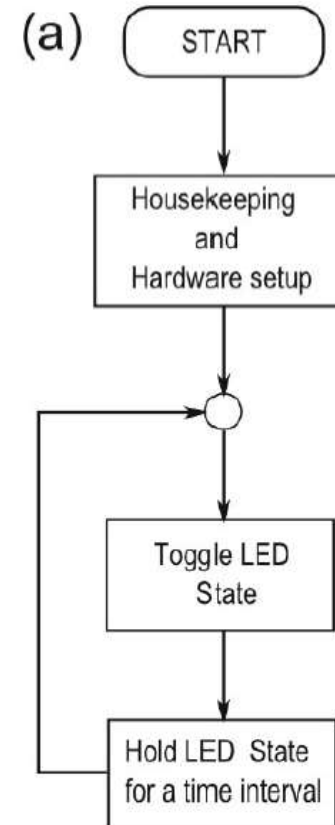
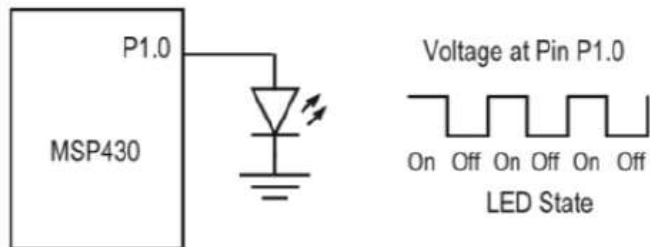
- High level language

- Preferred by most programmers because of its English like syntax and notation.

Programming

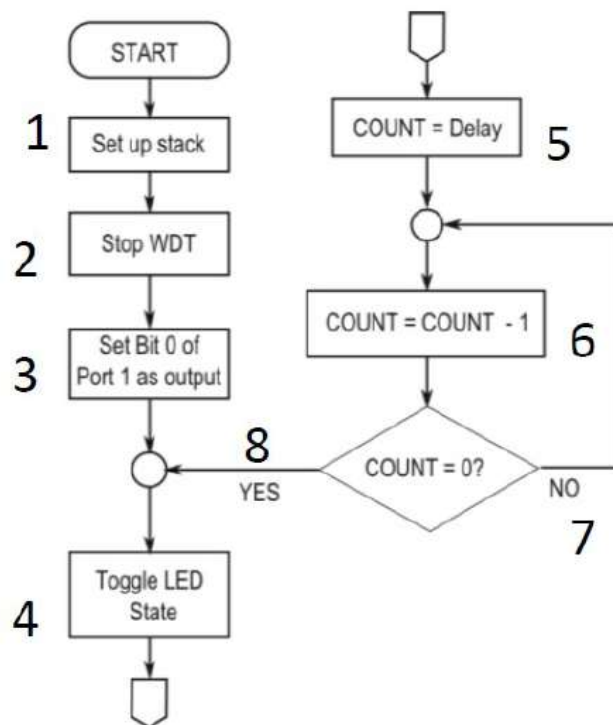
Example: Blinking LED Program

Objective: blink led by toggling of voltage levels



Programming

Programming Machine Language



Code! ☹

```

1: 4031 0300
2: 40B2 5A80 0120
3: D3D2 0022
4: E3D2 0021
5: 403F C350
6: 831F
7: 23FE
8: 3FF9
  
```

Programming

Programming Assembly Language

mov.w	#0x300, SP	↔	4031	0300	
mov.w	#0x5A80, &0x0120	↔	40B2	5A80	0120
bis.b	#001, &0x0022	↔	D3D2	0022	
xor.b	#001, &0x0021	↔	E3D2	0021	
mov.w	#0xC350, R15	↔	403F	C350	
dec.w	R15	↔	831F		
jnz	0x3FC	↔	23FE		
jmp	0x3F2	↔	3FF9		

There is a one-to-one correspondence between machine language and assembly language instructions – That's why disassembly!

Programming

Programming C Language

```
void main(void)
{
    WDTCTL = WDTPW | WDTHOLD;
    P1DIR |= 0x01;
    volatile unsigned int i;
    while(1)
    {
        P1OUT ^= 0x01;
        for(i=866; i>0; i--);    // delay
    }
}
```

Programming

Aspects of C for Embedded Systems

Programs for small embedded systems tend not to contain a lot of complicated manipulation of complex data objects.

Instead, much code is usually devoted to the control of peripherals through their special registers.

This means that the details of individual bits, bytes, and words are often important.

Declarations

The `const` and `volatile` qualifications are often critical.

const: Means that the value should not be modified: it is constant

Programming

volatile: For a variable that may appear to change “spontaneously,” with no direct action by the user’s program.

The compiler must therefore not keep a copy of the variable in a register for efficiency (like a cache).

Nor can the compiler assume that the variable remains constant when it optimizes the structure of the program—rearranging loops, for instance.

If the compiler did either of these, the program might miss externally induced changes to the contents of the memory associated with the variable

Programming

Low-Level Logic Operations

It is essential to distinguish between the Boolean logic operations and their bitwise counterparts:

- The Boolean form, such as the AND operation in `if (A && B)`, treats A and B as single Boolean values. Typically `A = 0` means false and `A != 0` means true. (A `bool` type in C++ and C99 can be used by including the header file `stdbool.h`.)

- In contrast, the bitwise operator `&` acts on each corresponding pair of bits in A and B individually. This means that eight operations are carried out in parallel if A and B are bytes.

For example, if `A = 10101010` and `B = 11001100`, then `A & B = 10001000`.

Similarly, the bitwise ordinary (inclusive) OR operation gives

`A | B = 11101110` and exclusive-or (XOR) gives `A ^ B = 01100110`.

Programming

Masks to Test Individual Bits

The smallest entity that can be handled by the CPU is a byte but it is often necessary to test or modify individual **bits** in an embedded system.

This can be done with bitwise logic operations.

Suppose that we want to know the value of bit 3 on the input of port 1, for instance. This means bit 3 of the byte P1IN, abbreviated to P1IN.3, for which we can use the standard definition $\text{BIT3} = 00001000$.

A pattern used for selecting bits is called a mask and has all zeroes except for a 1 in the position that we want to select.

Programming

Now consider the bitwise AND operation $\text{TestP13} = \text{P1IN} \& \text{BIT3}$.

All bits of TestP13 will be zero except for bit 3 because $x \& 0 = 0$ for either value of the bit x .

Bit 3 of TestP13 follows from $x \& 1 = x$ and is therefore made equal to bit 3 of P1IN.

Overall, TestP13 is zero if $\text{P1IN}.3 = 0$ and nonzero if $\text{P1IN}.3 \neq 0$. This is often used in decisions

```
if ((P1IN & BIT3) == 0) // Test P1.3 The () -> rules for precedence in C.
{
    // Actions for P1.3 == 0
}
else
{
    // Actions for P1.3 != 0
}
```

Programming

Masks to Modify Individual Bits

Masks can also be used to modify the value of individual bits.

Inclusive OR operation. Its truth table can be expressed as $x \mid 0 = x$ and $x \mid 1 = 1$. In words, taking the OR of a bit with 0 leaves its value unchanged, while taking its OR with 1 sets it to 1.

Thus we could set (force to 1) bit 3 of port 1 with **$P1OUT = P1OUT \mid BIT3$** , which leaves the values of the other bits unchanged.

This is usually abbreviated to **$P1OUT \mid= BIT3$** .

Programming

Masks to Modify Individual Bits

Clearing a bit (to 0) is done using AND, but is slightly more tricky.

The truth table can be expressed as $x \& 0 = 0$ and $x \& 1 = x$.

Thus we AND a bit with 1 to leave it unchanged and clear it with 0.

In this case, the mask should have **all ones** except for the bit that we wish to clear, so we use $\sim \text{BIT3}$ rather than BIT3 .

Thus we clear P1OUT.3 with $\text{P1OUT} \&= \sim \text{BIT3}$.

Finally, bits can be toggled (changed from 0 to 1 or 1 to 0) using the exclusive-or operation. For example, $\text{P1OUT} \hat{=} \text{BIT3}$ toggles P1OUT.3 .

Programming

Bit Fields

Bit fields resemble structures except that the number of bits occupied by each member is specified. For example the Timer_AControl register TACTL __no_init volatile union {

```
unsigned short TACTL; // Timer_A Control
struct {
    unsigned short TAIFG : 1;      // Timer_A counter interrupt flag
    unsigned short TAIE : 1;      // Timer_A counter interrupt enable
    unsigned short TACLR : 1;      // Timer_A counter clear
    unsigned short : 1;
    unsigned short TAMC : 2;       // Timer_A mode control
    unsigned short TAID : 2;       // Timer_A clock input divider
    unsigned short TASSEL : 2; // Timer_A clock source select
    unsigned short : 6;
} TACTL_bit;
} @ 0x0160;
```

Programming

The number after the colon shows the number of bits in the field. Each field often holds a single bit but there are several exceptions here; the mode is controlled by the pair of bits TAMC, for instance.

The unimplemented bits of the register must be included in the structure to keep the alignment correct but do not need names.

Now we can use a field instead of a mask to test the TAIFG bit with `if (TACTL_bit.TAIFG == 0)`.

Similarly, this bit can be

- Set with `TACTL_bit.TAIFG = 1`.
- Cleared with `TACTL_bit.TAIFG = 0`.
- Toggled with `TACTL_bit.TAIFG ^= 1`.

MSP430.h (part)

```
/* *****  
 * STANDARD BITS  
***** */  
  
#define BIT0          (0x0001)  
#define BIT1          (0x0002)  
#define BIT2          (0x0004)  
#define BIT3          (0x0008)  
#define BIT4          (0x0010)  
#define BIT5          (0x0020)  
#define BIT6          (0x0040)  
#define BIT7          (0x0080)  
#define BIT8          (0x0100)  
#define BIT9          (0x0200)  
#define BITA          (0x0400)  
#define BITB          (0x0800)  
#define BITC          (0x1000)  
#define BITD          (0x2000)  
#define BITE          (0x4000)  
#define BITF          (0x8000)
```

MSP430G2553 memory

Table 1. Available Options⁽¹⁾⁽²⁾

Device	BSL	EEM	Flash (KB)	RAM (B)	Timer_A	COMP_A+ Channel	ADC10 Channel	USCI_A0, USCI_B0	Clock	I/O	Package Type
MSP430G2553IRHB32	1	1	16	512	2x TA3	8	8	1	LF, DCO, VLO	24	32-QFN
MSP430G2553IPW28										24	28-TSSOP
MSP430G2553IPW20										16	20-TSSOP
MSP430G2553IN20										16	20-PDIP
MSP430G2453IRHB32	1	1	8	512	2x TA3	8	8	1	LF, DCO, VLO	24	32-QFN
MSP430G2453IPW28										24	28-TSSOP
MSP430G2453IPW20										16	20-TSSOP
MSP430G2453IN20										16	20-PDIP
-----										--	-----

MSP430G2553 memory

Memory Organization

Table 8. Memory Organization

		MSP430G2153	MSP430G2253 MSP430G2213	MSP430G2353 MSP430G2313	MSP430G2453 MSP430G2413	MSP430G2553 MSP430G2513
Memory	Size	1kB	2kB	4kB	8kB	16kB
Main: interrupt vector	Flash	0xFFFF to 0xFFC0	0xFFFF to 0xFFC0	0xFFFF to 0xFFC0	0xFFFF to 0xFFC0	0xFFFF to 0xFFC0
Main: code memory	Flash	0xFFFF to 0xFC00	0xFFFF to 0xF800	0xFFFF to 0xF000	0xFFFF to 0xE000	0xFFFF to 0xC000
Information memory	Size	256 Byte	256 Byte	256 Byte	256 Byte	256 Byte
	Flash	010FFh to 01000h	010FFh to 01000h	010FFh to 01000h	010FFh to 01000h	010FFh to 01000h
RAM	Size	256 Byte	256 Byte	256 Byte	512 Byte	512 Byte
		0x02FF to 0x0200	0x02FF to 0x0200	0x02FF to 0x0200	0x03FF to 0x0200	0x03FF to 0x0200
Peripherals	16-bit	01FFh to 0100h	01FFh to 0100h	01FFh to 0100h	01FFh to 0100h	01FFh to 0100h
	8-bit	0FFh to 010h	0FFh to 010h	0FFh to 010h	0FFh to 010h	0FFh to 010h
	8-bit SFR	0Fh to 00h	0Fh to 00h	0Fh to 00h	0Fh to 00h	0Fh to 00h

MSP430G2553 memory

MSP430G2553 MSP430G2513
16kB 0xFFFF to 0xFFC0 0xFFFF to 0xC000
256 Byte 010FFh to 01000h
512 Byte 0x03FF to 0x0200
01FFh to 0100h 0FFh to 010h 0Fh to 00h

Does it add up?

MSP430G2553 memory

MSP430G2553 MSP430G2513
16kB 0xFFFF to 0xFFC0 0xFFFF to 0xC000
256 Byte 010FFh to 01000h
512 Byte 0x03FF to 0x0200
01FFh to 0100h 0FFh to 010h 0Fh to 00h

Does it add up?

FFFF-3FFF=C000 start of code in Flash

RAM starts at 0200 for user variables

Don't want to overwrite the SFRs and peripheral registers

256 bytes of information memory between flash and RAM

Stack ideally intilized at top of RAM



THANK YOU

Rex Joseph

Department of Electrical and Electronics Engineering

rexjoseph@pes.edu

+91 80 6666 3333 Extn 515